

LZW 字典编码(压缩 + 解压)

Huffman 靠"字符出现频率"省位;LZW 换一个思路——它一边读一边把反复出现的字符串攒进一本字典,后面再遇到就只发一个短码。字典不用随文件传输,因为解压端会照同样的规则把它重新长出来。

直觉类比

想象你和朋友发消息,反复要写"操作系统原理"。第一次你写全称,并约定"以后这个词就叫①"。之后你只发"①",对方查约定表就知道是什么。LZW 就是这样:压缩端每发现一个新的重复片段,就给它编一个号存进字典;而解压端只要按完全相同的规则,也能把这本字典一个字不差地重建出来——所以字典本身根本不用发送。这就是"动态字典"的精髓。

1. 核心定义

LZW (Lempel-Ziv-Welch)

一种无损的、基于字典的压缩算法。用定长的码字去代表那些经常一起出现的变长字符串。GIF、TIFF、ZIP/PKZIP、WinRAR 等都用它。关键: 定长码字 $\leftrightarrow$ 变长字符串。

字典 / 串表 (Dictionary / String Table)

一张"字符串 $\rightarrow$ 码字"的对照表。压缩和解压双方各自动态维护一份,并保证内容完全一致——所以字典不随数据传输。

初始字典

开始前先把所有单个字符放进字典,给定初始码字。本讲例子里只有三个字符: A=1, B=2, C=3。新码字从 4 开始递增。

2. 压缩算法(背下这段伪代码)

整个 LZW 压缩就是这段循环,考试时手推全靠它。s 是"当前已确认在字典里的前缀", c 是"下一个字符":

伪代码	大白话
s = 第一个字符	先拿第一个字符当前缀
c = 下一个字符	再读一个字符
if (s+c) 在字典里: s = s+c	拼起来还认识 $\rightarrow$ 把前缀加长,继续贪心往后吃,不输出
else: 输出 s 的码字; 把 s+c 加进字典(给新码); s = c	拼起来不认识 $\rightarrow$ 输出旧前缀 s 的码;把这个新串 s+c 记进字典;前缀重置为 c
末尾: 输出 s 的码字	循环结束别忘了把最后攒的 s 也发出去

一句话记忆:能拼长就拼长(不输出),拼不动了就吐旧码 + 记新串 + 从当前字符重来。
字典里新加的永远是"已输出的串 + 下一个字符",即比刚吐出去的那段多一个字符。

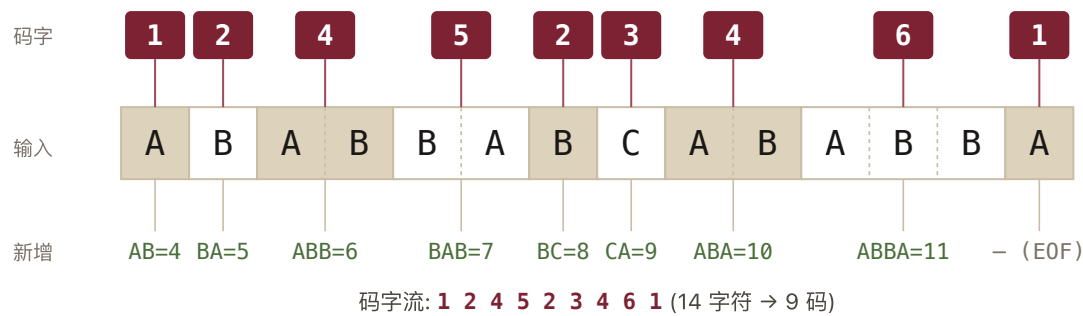
3. 例题(核心):对 **ABABBABCABABBA** 做 **LZW**

初始字典 A=1, B=2, C=3 ,新码从 4 起。逐字符处理,每一行看 s+c 是否在字典里:

步	S (当前前缀)	C (读入)	S+C 在字典?	输出码	新增字典项	新 S
1	A	B	否 (AB)	1	AB=4	B
2	B	A	否 (BA)	2	BA=5	A
3	A	B	是 (AB=4)	—	—	AB
4	AB	B	否 (ABB)	4	ABB=6	B
5	B	A	是 (BA=5)	—	—	BA
6	BA	B	否 (BAB)	5	BAB=7	B
7	B	C	否 (BC)	2	BC=8	C
8	C	A	否 (CA)	3	CA=9	A
9	A	B	是 (AB=4)	—	—	AB
10	AB	A	否 (ABA)	4	ABA=10	A
11	A	B	是 (AB=4)	—	—	AB
12	AB	B	是 (ABB=6)	—	—	ABB
13	ABB	A	否 (ABBA)	6	ABBA=11	A
末	A	EOF	—	1	—	—

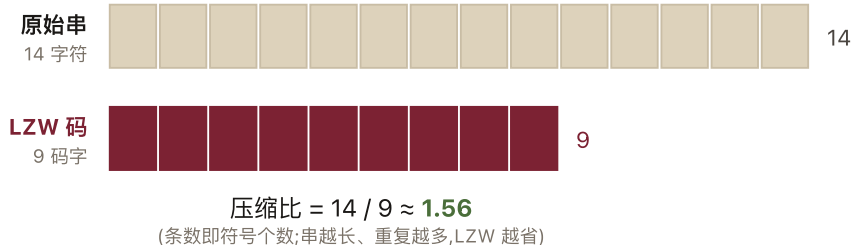
输出码序列:1 2 4 5 2 3 4 6 1(共 9 个码)。原串 14 个字符 → 9 个码,压缩比 = 14 / 9 ≈ 1.56。

压缩:输入怎样被切成短语并吐码



贪心切分。算法把输入串从左到右切成一段段"已在字典里的最长短语"(底部虚线为短语内的单字符),每切出一段就**吐一个码字**(顶部红块),同时把"该短语 + 下一个字符"登记为新字典项(下方绿字,码号 4→11 连续递增)。EOF 处最后一段 A 无后继字符,故不再新增字典项。

压缩效果:符号个数对比



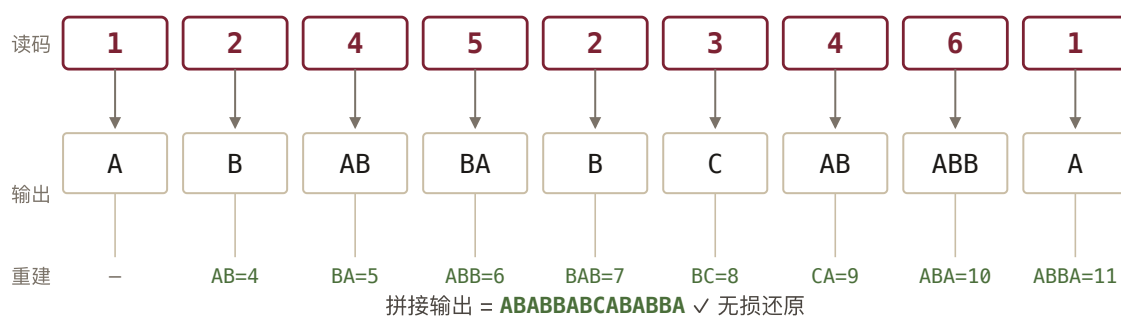
怎么读。每个方块是一个待存储的符号:原始串 14 个字符,LZW 后 9 个码字。**条数越短越省。**注意压缩比按"符号个数"算得 1.56;真实比特数还取决于码字位宽(码字定长、通常比一个字节的 8 位更宽),故短串未必真省——LZW 的收益随**串越长、重复片段越多**而放大。

解压端拿到 1 2 4 5 2 3 4 6 1,初始字典同样是 A=1, B=2, C=3 。规则: 每读一个码 k ,先输出它对应的串 entry ;若上一步的 s 不为空,就把 s + entry[0] (s 加 entry 的首字符)记进字典;然后 s = entry 。

步	S (上次串)	K (读码)	ENTRY=输出	新增字典项 S+ENTRY[0]	新 S
1	NIL	1	A	—(s 为空,不加)	A
2	A	2	B	A+B = AB=4	B
3	B	4	AB	B+A = BA=5	AB
4	AB	5	BA	AB+B = ABB=6	BA
5	BA	2	B	BA+B = BAB=7	B
6	B	3	C	B+C = BC=8	C
7	C	4	AB	C+A = CA=9	AB
8	AB	6	ABB	AB+A = ABA=10	ABB
9	ABB	1	A	ABB+A = ABBA=11	A

拼接所有 entry:**A · B · AB · BA · B · C · AB · ABB · A = ABABBABCABABBA**,与原串完全一致——无损还原成功。注意解压端重建出的字典 (AB=4, BA=5, ABB=6, BAB=7, BC=8, CA=9, ABA=10, ABBA=11) 与压缩端一模一样,这正是字典无需传输的原因。

解压:码字流如何重建原串与字典



对称还原。解压端依次读码(红框),每个码查表输出对应短语(灰框),并把"上次输出的串 + 本次短语的首字符"登记为新字典项(下方绿字)。重建出的字典 (AB=4...ABBA=11) 与压缩端**逐项一致**——这正是字典无需随数据传输的原因。

压缩与解压的对称性:两边都在"输出一个已知串之后,才把'该串 + 下一个字符'加入字典"。压缩端的"下一个字符"是它当前正读到的 c;解压端的"下一个字符"是下一个 entry 的首字符。规则同源,字典自然同步。

4. 手推 LZW 的三个易错点

- 输出的是"旧前缀"的码,不是拼接后的码。拼不动时先吐 s 的码,再登记 s+c。很多人错把新串的码当场输出了——但新串刚建,这一步不会用到它。
- 新码字连续递增,别漏号也别跳号。从 4、5、6...依次给,顺序就是"字典项被创建的先后"。
- 末尾那次输出别忘。循环靠 EOF 结束,但退出前 s 里还攒着东西,必须补输出一次(本例末尾的那个 1)。

自测题

初始字典为 A=1, B=2 (新码从 3 起)。对输入串 ABABABA 做 LZW 压缩,输出的码序列是?

- A. 1 2 3 4 5
- B. 1 2 3 5
- C. 1 2 4 5
- D. 1 3 5 2

▼ 答案与解释

答案:B → 1 2 3 5

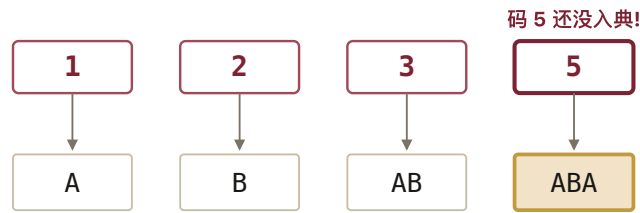
逐步手推(s=前缀, c=读入):

- s=A, c=B:AB 不在 → 输出 **1**,加 AB=3,s=B
- s=B, c=A:BA 不在 → 输出 **2**,加 BA=4,s=A
- s=A, c=B:AB 在(3)→ s=AB
- s=AB, c=A:ABA 不在 → 输出 **3**,加 ABA=5,s=A
- s=A, c=B:AB 在(3)→ s=AB
- s=AB, c=A:ABA 在(5)→ s=ABA
- EOF → 输出 **5**(即 ABA 的码)

所以输出 **1 2 3 5**:7 个字符压成 4 个码。

陷阱:选 A 的人多输出了一个码,正是忘了"拼得动就继续吃、不输出";末尾 ABA 已成串,只发它的码 5,而不是拆成 3、1。

经典陷阱:解压端的异常情形 (以本页自测题 ABABABA 为例)



异常情形 (cScSc):当前码 = 即将新建的字典号

规则:entry = 上次串 s + s 的首字符。此处 s=AB → entry = AB + A = **ABA**

cScSc 情形。把 ABABABA 压成码流 1 2 3 5 后解压,读到第 4 个码 **5** 时字典里还没有 5 号项——这是唯一的异常:**当前码正好等于即将新建的字典号**。此时不能查表,改用公式 entry = 上次串 s + s 的首字符,即 AB + A = ABA,再把它登记为 5 号。凡是"某字符串紧接着以自己开头重复出现"就会触发,考试务必单独检查这一步。

下一步 → 进入 0012 · 算术编码(区间细分),换一种把"整条消息压成一个小数区间"的熵编码,真题里和 LZW 一样是高频计算题。